```
"""Upload a video to one of our YouTube channels through our own Google Cloud app (Knacktoon
Uploader).

Standard library only. The app's key is secrets/client_secret.json; each channel's sign-in is kept in
secrets/token_<label>.json (both git-ignored, never printed).

  python scripts/yt_upload.py auth en          # once per channel: opens Google sign-in in your
browser; pick the channel
  python scripts/yt_upload.py whoami en        # which channel a sign-in belongs to
  python scripts/yt_upload.py upload en out/Bees/Bees.captions.mp4 --title "..." --description "..."
--lang en
         [--privacy private|unlisted|public] [--publish-at 2026-11-05T17:00:00-05:00] [--category 27]

Until Google audits the project, YouTube locks every upload from it as private (docs/CHANNELS.md).
--publish-at schedules the release (YouTube needs the video private until then).
"""
import argparse, base64, hashlib, http.server, json, os, secrets as rnd, sys, time, urllib.parse,
urllib.request, webbrowser

ROOT = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "secrets")
SCOPES = "https://www.googleapis.com/auth/youtube.upload
https://www.googleapis.com/auth/youtube.readonly"


def client():
    return json.load(open(os.path.join(ROOT, "client_secret.json")))["installed"]


def post_form(url, data):
    req = urllib.request.Request(url, urllib.parse.urlencode(data).encode(), {"Content-Type":
"application/x-www-form-urlencoded"})
    return json.load(urllib.request.urlopen(req))


def auth(label):
    c = client()
    verifier = base64.urlsafe_b64encode(rnd.token_bytes(40)).rstrip(b"=").decode()
    challenge =
base64.urlsafe_b64encode(hashlib.sha256(verifier.encode()).digest()).rstrip(b"=").decode()
    state, got = rnd.token_urlsafe(16), {}

    class H(http.server.BaseHTTPRequestHandler):
        def do_GET(self):
            q = dict(urllib.parse.parse_qsl(urllib.parse.urlparse(self.path).query))
            if q.get("state") == state: got.update(q)
            self.send_response(200); self.send_header("Content-Type", "text/html; charset=utf-8");
self.end_headers()
            self.wfile.write("<h2>Knacktoon Uploader: signed in. You can close this tab.
</h2>".encode())
        def log_message(self, *a): pass

    srv = http.server.HTTPServer(("127.0.0.1", 0), H)
    redirect = f"http://127.0.0.1:{srv.server_port}"
    url = "https://accounts.google.com/o/oauth2/v2/auth?" + urllib.parse.urlencode({
        "client_id": c["client_id"], "redirect_uri": redirect, "response_type": "code", "scope":
SCOPES, "state": state,
        "access_type": "offline", "prompt": "consent select_account", "code_challenge": challenge,
"code_challenge_method": "S256"})
    print(f"Sign in for '{label}' in the browser window that opens, and choose that channel. If
nothing opens, visit:\n{url}", flush=True)
    webbrowser.open(url)
    while "code" not in got and "error" not in got:
        srv.handle_request()
    if "error" in got: sys.exit(f"sign-in failed: {got['error']}")
```

```
    tok = post_form(c["token_uri"], {"code": got["code"], "client_id": c["client_id"],
"client_secret": c["client_secret"],
                                     "redirect_uri": redirect, "grant_type": "authorization_code",
"code_verifier": verifier})
    if "refresh_token" not in tok: sys.exit("Google returned no refresh token; run auth again")
    json.dump({"refresh_token": tok["refresh_token"]}, open(os.path.join(ROOT,
f"token_{label}.json"), "w"))
    print("saved sign-in for", label, "->", whoami(label))


def access_token(label):
    c, t = client(), json.load(open(os.path.join(ROOT, f"token_{label}.json")))
    return post_form(c["token_uri"], {"refresh_token": t["refresh_token"], "client_id":
c["client_id"],
                                      "client_secret": c["client_secret"], "grant_type":
"refresh_token"})["access_token"]


def api(label, url, data=None, headers=None, method=None):
    h = {"Authorization": "Bearer " + access_token(label), **(headers or {})}
    return urllib.request.urlopen(urllib.request.Request(url, data, h, method=method))


def whoami(label):
    r = json.load(api(label, "https://www.googleapis.com/youtube/v3/channels?
part=snippet&mine=true"))
    return ", ".join(f"{i['snippet']['title']} ({i['id']})" for i in r.get("items", [])) or "no
channel"


def upload(a):
    body = {"snippet": {"title": a.title, "description": a.description, "categoryId": a.category,
                        "defaultLanguage": a.lang, "defaultAudioLanguage": a.lang},
            "status": {"privacyStatus": "private" if a.publish_at else a.privacy,
"selfDeclaredMadeForKids": False,
                       "embeddable": True, "license": "youtube"}}
    if a.publish_at: body["status"]["publishAt"] = a.publish_at
    size = os.path.getsize(a.file)
    start = api(a.label, "https://www.googleapis.com/upload/youtube/v3/videos?
uploadType=resumable&part=snippet,status",
                json.dumps(body).encode(), {"Content-Type": "application/json; charset=UTF-8",
                                            "X-Upload-Content-Type": "video/mp4", "X-Upload-Content-
Length": str(size)}, "POST")
    session = start.headers["Location"]
    with open(a.file, "rb") as f:
        data = f.read()
    for attempt in range(5):
        try:
            r = json.load(urllib.request.urlopen(urllib.request.Request(session, data, {"Content-
Type": "video/mp4"}, method="PUT")))
            break
        except urllib.error.HTTPError as e:
            if e.code < 500 or attempt == 4: sys.exit(f"upload failed: {e.code} {e.read()[:300]}")
            time.sleep(2 ** attempt)
    print(json.dumps({"id": r["id"], "url": f"https://youtube.com/shorts/{r['id']}", "privacy":
r["status"]["privacyStatus"],
                      "publishAt": r["status"].get("publishAt"), "uploadStatus":
r["status"].get("uploadStatus")}))


p = argparse.ArgumentParser()
sub = p.add_subparsers(dest="cmd", required=True)
sub.add_parser("auth").add_argument("label")
sub.add_parser("whoami").add_argument("label")
u = sub.add_parser("upload")
```

```
u.add_argument("label"); u.add_argument("file")
u.add_argument("--title", required=True); u.add_argument("--description", default="")
u.add_argument("--lang", default="en"); u.add_argument("--category", default="27")
u.add_argument("--privacy", default="private", choices=["private", "unlisted", "public"])
u.add_argument("--publish-at")
a = p.parse_args()
if a.cmd == "auth": auth(a.label)
elif a.cmd == "whoami": print(whoami(a.label))
else: upload(a)
```